

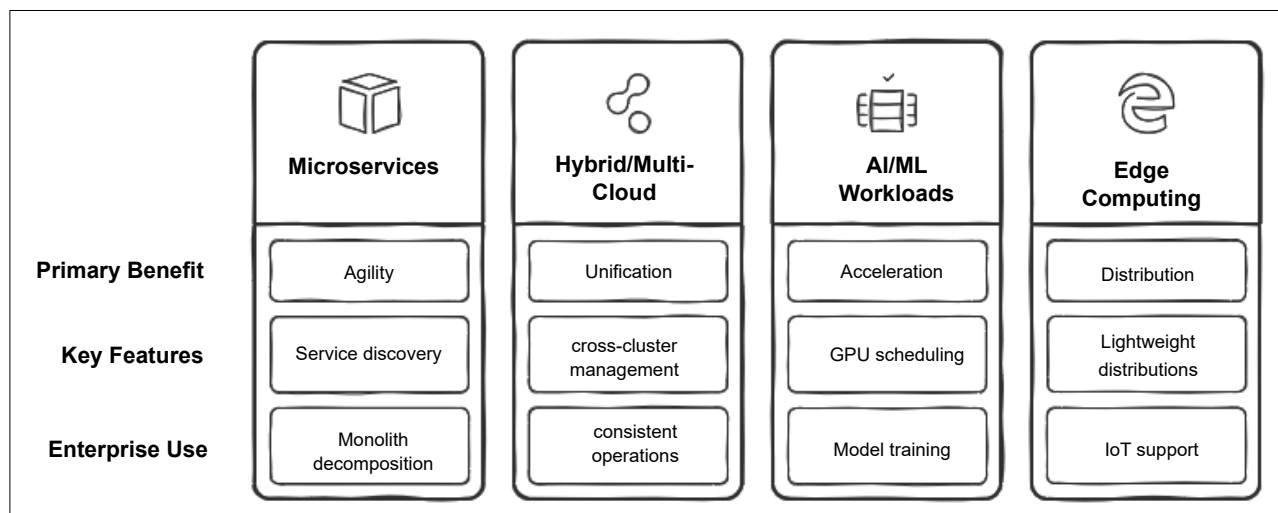


Figure 2: Kubernetes use cases in the enterprise

Architectural considerations

When adopting Kubernetes in the enterprise, architectural considerations play a pivotal role in ensuring the platform is scalable, secure, and aligned with organisational goals. At the heart of Kubernetes architecture is the separation between the control plane and the worker nodes. The control plane is responsible for maintaining the desired state of the cluster, managing scheduling, scaling, and health monitoring, while the worker nodes run the actual application workloads. In enterprise environments, it is common to deploy the control plane in a high-availability configuration across multiple zones or regions to ensure resilience and fault tolerance. This setup is especially critical for organisations that rely on Kubernetes to host customer-facing or mission-critical applications.

Another key architectural element is the integration of a service mesh, such as Istio or Linkerd. These tools provide advanced traffic management, observability, and security features that are essential in complex microservices environments. For example, service meshes enable fine-grained control over how services communicate, enforce policies, and collect telemetry data. This is particularly valuable in regulated industries where compliance and auditability are non-negotiable. By

abstracting these concerns away from application code, service meshes help maintain clean separation of concerns and reduce the operational burden on development teams.

Enterprises must also consider how Kubernetes fits into their CI/CD pipelines. Kubernetes supports a wide range of open source tools that facilitate continuous integration and delivery, including ArgoCD, Tekton, and Jenkins X. These tools enable GitOps workflows, where the desired state of the system is defined in Git and automatically reconciled with the live environment. This approach not only improves deployment consistency and traceability but also aligns with modern DevOps practices. Architecting Kubernetes to support GitOps requires careful planning around repository structure, access control, and secrets management.

Observability is another cornerstone of Kubernetes architecture. Enterprises require comprehensive visibility into their clusters to ensure optimal performance, detect anomalies, and effectively troubleshoot issues. The *de facto* observability stack—comprising Prometheus for metrics, Grafana for visualisation, Fluentd for log aggregation, and Jaeger for distributed tracing—provides a robust foundation for monitoring Kubernetes

environments. Integrating these tools into the architecture from the outset enables teams to manage system health and optimise resource usage proactively. Moreover, observability data can be leveraged to inform automated scaling and alerting systems, thereby enhancing operational efficiency.

Security and governance must be embedded into the Kubernetes architecture from day one. RBAC, network policies, and secrets management are native features that help enforce security boundaries and protect sensitive data. Enterprises often extend these capabilities with tools like Open Policy Agent (OPA) for policy enforcement and HashiCorp Vault for secure secrets storage. Designing a secure Kubernetes architecture involves not just configuring these tools correctly but also establishing processes for auditing, compliance, and incident response. In large organisations, this often means integrating Kubernetes with centralised identity providers as well as security information and event management (SIEM) systems.

Ultimately, architectural decisions in Kubernetes are not one-size-fits-all. They must be tailored to the enterprise's specific needs, including workload characteristics, regulatory requirements, and